



DEVOPS
TOOLS ENGINEER

TOPIC-ELEVEN

Learning Prometheus



Arash Foroughi

[linkedin.com/in/arashforoughi/](https://www.linkedin.com/in/arashforoughi/)

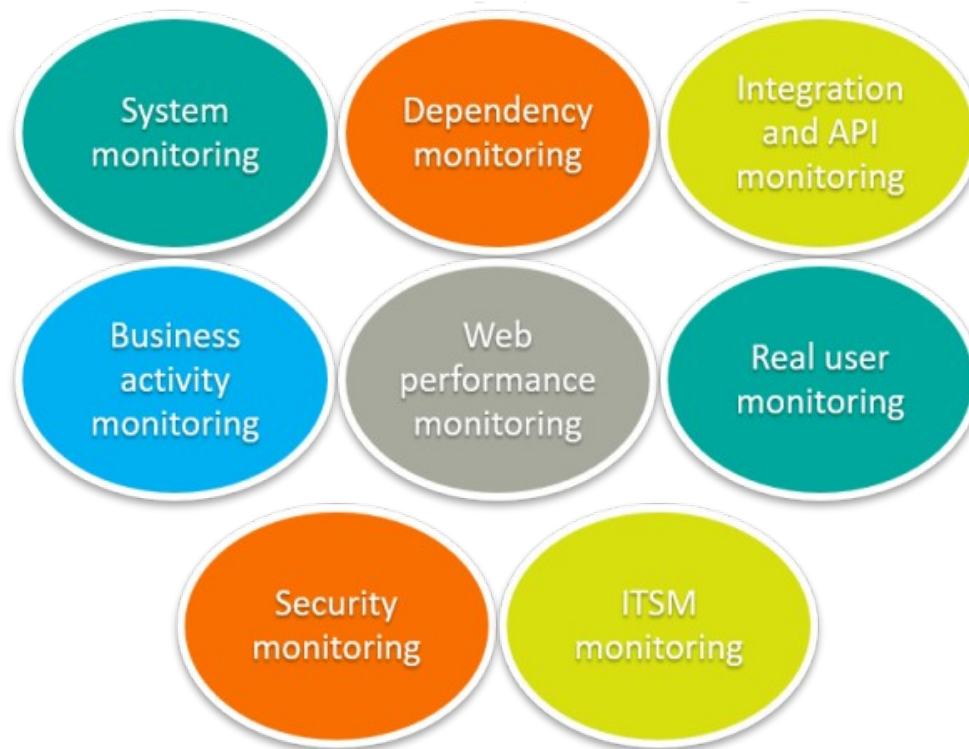
Monitoring Concepts



Monitoring Concepts

- IT monitoring is a **process to gather metrics** about the operations of an IT environment's **hardware & software** to ensure all functions as expected to support apps & services.
- **Basic monitoring** is performed through device operation checks, while **advanced monitoring** gives granular views on operational states, including average response times, number of application instances, error/request rates, CPU usage & application availability.
- IT monitoring can rely on **agents** or be **agentless**.
 - **Agents** are independent programs that install on the monitored device to collect data on hardware or software performance data and report it to a management server. (SNMP)
 - **Agentless monitoring** uses existing communication protocols to emulate an agent, with many of the same functionalities. (SSH, SSL)
- What is the difference between **Monitoring Tools** and **Log Analyzers**?!

Different Types of Monitoring Systems



Key Elements of a Monitoring System

- **Metrics**, **monitoring**, & **alerting** are the key elements of a monitoring system.
- **Metrics** are the input, the raw data needed for monitoring performance, health, and availability. **Monitoring** is what **alerting** is built on top of.
- Together, they detect **performance** or **usage** anomalies, help you troubleshoot and identify issues, reveal usage and behavior patterns or **trends**, and help you understand the impact of any changes you make to your applications or infrastructure.
- Whenever metrics meet **conditions** you have **defined** as part of **alert rules**, monitoring solutions send **notifications** prompting you to investigate issues and even help you narrow down and identify possible root causes.

What are System Metrics?

- Metrics are raw data about **resource usage** or behavior that monitoring system collects from any applications or services on infrastructure.
- This is typically done with a **monitoring agent**, but it can also be done without it for **serverless** applications.
- **Operating system metrics** refer to those metrics that provide information about the availability of resources such as **CPU, memory, disk space** or various **processes**. Other metrics, say for database monitoring, web servers, etc. are collected in a similar ways.
- With **metrics & events** can set up alert conditions to be notified when these rules are met.

What are Events & Logs?

□ Events

- Besides metrics, monitoring systems can also capture **events**. While metrics are collected on **fixed intervals**, events are typically generated at varying **frequencies**.
- Events typically capture what happened, where it happened, and when. They work hand in hand with metrics by providing a bit more context which helps find **Root Cause** faster.

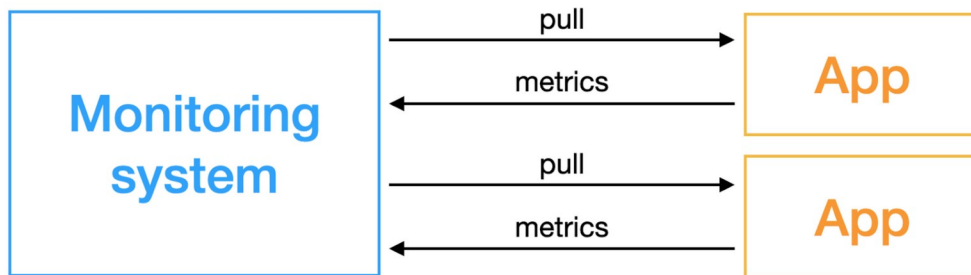
□ Logs

- While metrics describe application or services **trends** and current performance, logs provide **information** about what applications or services are and have been doing.
- They create a trail of events that show what happened, where, and when. This makes them extremely valuable for troubleshooting.

Pull-Based vs. Push-Based Monitorings

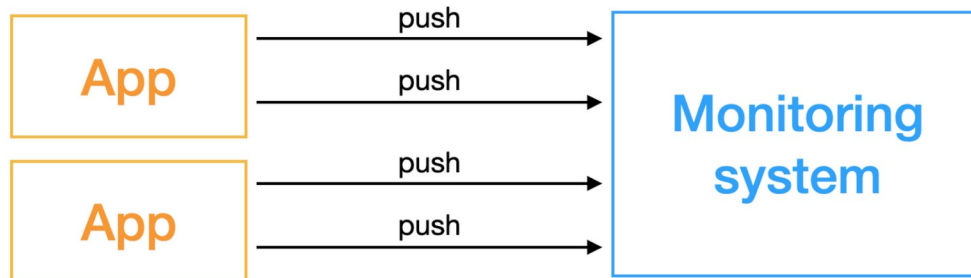


Pull-based system



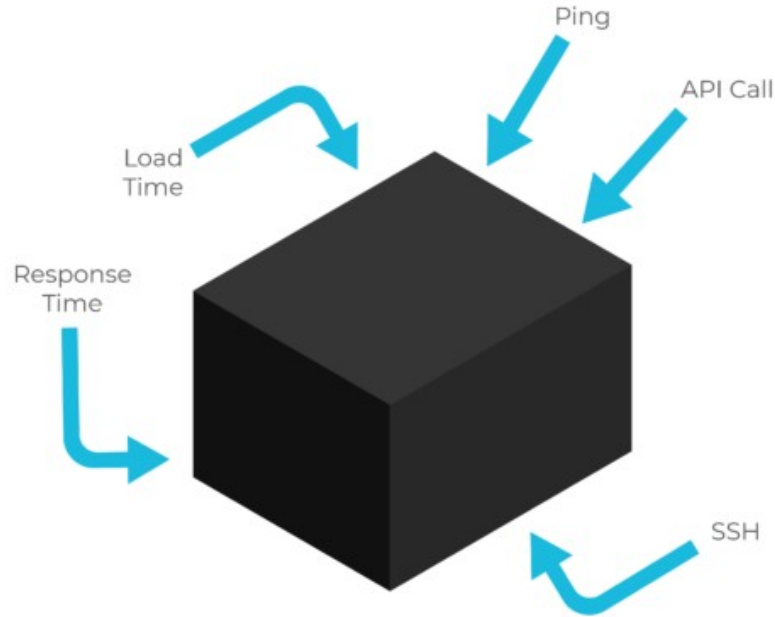
PASSIVE
Monitoring

Push-based monitoring system

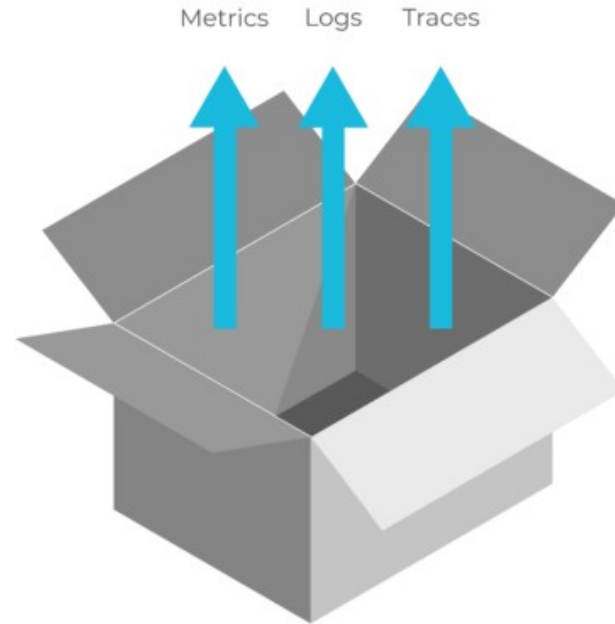


ACTIVE
Monitoring

Blackbox vs. Whitebox Monitoring



Blackbox Monitoring



Whitebox Monitoring

Blackbox vs. Whitebox Monitoring

CATEGORIZATION OF APPLICATION MONITORING TECHNIQUES

	INSTANCE-LEVEL (DIAGNOSTIC-ORIENTED MONITORING)	SERVICE-LEVEL (OBJECTIVE-ORIENTED MONITORING)
BLACK-BOX (NO CODE CHANGE REQUIRED)	- Using collectors such as Datadog agent, Nagios, Telegraf, etc. - Using operating system based insights such as Berkeley Packet Filter (BPF) Datadog, Nagios, Sysdig	Network packet capture leveraging Pcap Netsil
WHITE-BOX (CODE CHANGE REQUIRED EITHER AT IMPLEMENTATION OR RUN TIME)	- Code instrumentation such as Java byte-code instrumentation - Custom metrics using Statsd - Log analysis New Relic, AppDynamics, Instana, Splunk, Sumo Logic	- Distributed tracing frameworks such as Google Dapper, Twitter Zipkin - Tracing using specialized proxies such as Linkerd Lightstep, Buoyant

Introduction of Prometheus



Introduction of Prometheus

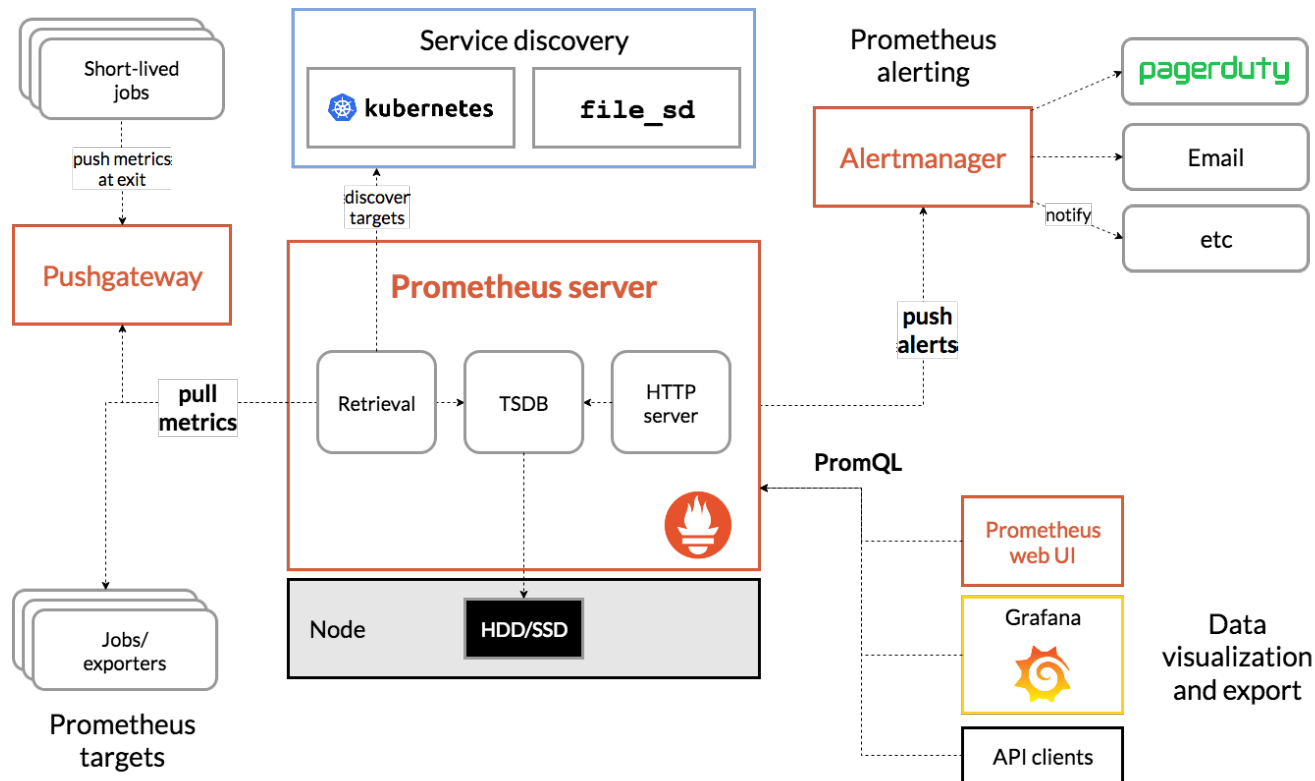
- In Greek mythology, **Prometheus** is a **Titan god of fire**. Prometheus is best known for defying the gods by stealing fire from them and giving it to humanity in the form of technology, knowledge, and more generally, civilization.
- **Prometheus** is an open-source **monitoring & alerting toolkit** originally built at **SoundCloud** in 2012 and written by **Go**.
- Prometheus joined the **Cloud Native Computing Foundation** in **2016** as the second hosted project, after Kubernetes.
- Prometheus collects and stores its **metrics** as time series data.
- **Exporter** is a piece of software can fetch statistics from another, non-Prometheus system & turn them into Prometheus metrics, using a client library.

Prometheus Features

❑ Prometheus's main features are:

- a multi-dimensional data model with time series data identified by metric name and key/value pairs
- PromQL, a flexible query language to leverage this dimensionality
- no reliance on distributed storage; single server nodes are autonomous
- time series collection happens via a pull model over HTTP
- pushing time series is supported via an intermediary gateway
- targets are discovered via service discovery or static configuration
- multiple modes of graphing and dashboarding support

Prometheus Architecture



Prometheus Architecture

- ❑ **Prometheus server** collects time series data, stores it, makes it available for querying, and sends alerts based on them.
- ❑ **AlertManager** receives alert triggers from Prometheus and handles routing and dispatching of alerts.
- ❑ **PushGateway** handles the exposition of metrics that have been pushed from short-lived jobs such as cron or batch jobs.
- ❑ **Applications** that support the Prometheus exposition format make internal state available through an HTTP endpoint.
- ❑ **Community-driven** exporters expose metrics from applications that do not support Prometheus natively.
- ❑ First-party & third-party dashboarding solutions provide a **visualization** of collected data.

Exporters



Exporters

- ❖ Not all code you run is code that you can control or even have access to, and thus adding direct instrumentation isn't really an option.
- ❖ An exporter is a piece of software that you deploy right beside the application you want to obtain metrics from.
- ❖ It takes in HTTP requests from Prometheus, gathers the required data from the application, transforms them into the correct format, and finally returns them in a response to Prometheus.
- ❖ Each exporter usually targets a specific service or application and as such, their deployment reflects this one-to-one synergy.
- ❖ If the exporter is missing a metric you are interested in, you can always send a pull request to improve it, making it better for the next person to use it.

Exporter Fundamentals

- ❑ When the exporter starts, it binds to a **configured port** and exposes the internal state of whatever is being collected in an **HTTP** endpoint of your choosing (the default being **/metrics**).
- ❑ The instrumentation data is collected when an **HTTP GET request** is made to the configured endpoint.
- ❑ The HTTP GET request that's made by the **Prometheus server** to the observed system for metric collection is called a **scrape**.

Prometheus Dashboards

- ❖ Prometheus has a number of HTTP APIs that allow you to both request raw data and evaluate **PromQL queries**.
- ❖ These can be used to produce graphs and dashboards.
- ❖ Out of the box, Prometheus provides the expression browser.
 - ❖ It uses these APIs and is suitable for ad hoc querying and data exploration, but it is not a general dashboard system.
- ❖ It is recommended that you use **Grafana** for dashboards.
 - ❖ It has wide variety of features, including official support for Prometheus as data source.

Setup Procedure



Setup Procedure

❑ Setup Procedure

- 1) Install Prometheus and Grafana
- 2) Install Prometheus Node Exporter on Linux servers to be monitored
- 3) Configure Node Exporter
- 4) Configure Prometheus server with Scrap jobs
- 5) Add Dashboards to Grafana
- 6) Start visualizing system metrics on Grafana

Install Prometheus & Grafana



I. Create Prometheus System Group

- ❑ Other tools which make Prometheus complete monitoring tool are:
 - **Exporters:** These are libraries that help with exporting metrics from third-party systems as Prometheus metrics.
 - **PromQL:** Prometheus query language which allows you to filter multi-dimensional time series data.
- Let's start by creating the Prometheus system user and group.

```
root@ubuntu:~# groupadd --system prometheus
```

- The group with ID < 1000 is a system group. Once the system group is added, create Prometheus system user and assign primary group created.

```
root@ubuntu:~# useradd -s /sbin/nologin --system -g prometheus prometheus
```

<https://prometheus.io/download/>

II. Create Data & Config Directories

- Prometheus needs a directory to store its data. Create it under **/var/lib/prometheus**.

```
root@ubuntu:~# mkdir /var/lib/prometheus
```

- Prometheus primary configuration files directory is **/etc/prometheus/**. It will have some sub-directories:

```
root@ubuntu:~# for i in rules rules.d files_sd; do sudo mkdir -p  
/etc/prometheus/${i}; done
```

III. Download Prometheus Tar File

- We need to download the latest release of Prometheus archive and extract it to get binary files. You can check releases from [Prometheus releases Github](#) page.

```
root@ubuntu:~# sudo apt update; sudo apt -y install wget curl vim
root@ubuntu:~# mkdir -p /tmp/prometheus && cd /tmp/prometheus
root@ubuntu:~# curl -s
https://api.github.com/repos/prometheus/prometheus/releases/latest | grep
browser_download_url | grep linux-amd64 | cut -d '"' -f 4 | wget -qi -
root@ubuntu:~# tar xvf prometheus*.tar.gz
root@ubuntu:~# cd prometheus*/
root@ubuntu:~# sudo mv prometheus promtool /usr/local/bin/
root@ubuntu:~# sudo mv prometheus.yml console* /etc/prometheus/
root@ubuntu:~# prometheus --version; promtool --version
```

IV. Configure Prometheus on Ubuntu

- Change the ownership of these directories to Prometheus user and group.

```
root@ubuntu:~# cd /etc/prometheus
root@ubuntu:~# chown -R prometheus:prometheus rules.d files_sd
root@ubuntu:~# chmod -R 775 rules.d files_sd
root@ubuntu:~# chown -R prometheus:prometheus /var/lib/prometheus
```

- Create or edit a configuration file for Prometheus – **/etc/prometheus/prometheus.yml**.

```
root@ubuntu:~# sudo vim /etc/prometheus/prometheus.yml
root@ubuntu:~# sudo vim /etc/systemd/system/prometheus.service
root@ubuntu:~# sudo systemctl daemon-reload
root@ubuntu:~# sudo systemctl enable --now prometheus
```



Prometheus.yml prometheus.servi
ce

Welcome to Prometheus!!

The screenshot shows the Prometheus web interface in a browser. The address bar indicates the URL is `192.168.18.9:9090/graph`. The interface includes a navigation bar with links for Alerts, Graph, Status, and Help. A checkbox for 'Enable query history' is present. The main query editor contains a text input field with the placeholder 'Expression (press Shift+Enter for newlines)'. Below the input field is an 'Execute' button and a dropdown menu currently showing '- insert metric at cursor'. Two tabs, 'Graph' and 'Console', are visible, with 'Graph' being the active tab. The graph area features a 'Moment' selector with left and right arrow buttons. Below this is a table with two columns: 'Element' and 'Value'. The table is currently empty, displaying 'no data'. In the bottom right corner of the graph area, there is a 'Remove Graph' link. At the bottom left, there is an 'Add Graph' button.

Not secure 192.168.18.9:9090/graph

Prometheus Alerts Graph Status ▾ Help

☐ Enable query history

Expression (press Shift+Enter for newlines)

Execute - insert metric at cursor ▾

Graph Console

◀ Moment ▶

Element	Value
no data	

Remove Graph

Add Graph

Install Grafana Using APT Repository

- Install Grafana 8 from APT Repository:

```
root@ubuntu:~# apt update
root@ubuntu:~# apt install -y gnupg2 curl software-properties-common
root@ubuntu:~# curl https://packages.grafana.com/gpg.key | sudo apt-key
add -
root@ubuntu:~# add-apt-repository "deb
https://packages.grafana.com/oss/deb stable main"
root@ubuntu:~# apt update; apt install grafana -y
root@ubuntu:~# systemctl enable --now grafana-server
root@ubuntu:~# ufw allow 3000/tcp; ufw reload; systemctl
restart ufw
```

Access Grafana Dashboard

- Access Grafana Dashboard using **IP:3000** and default logins of **admin/admin**.

❑ GRAFANA PACKAGE DETAILS:

- Installs binary to **/usr/sbin/grafana-server**
- Installs Init.d script to **/etc/init.d/grafana-server**
- Creates default file (environment vars) to **/etc/default/grafana-server**
- Installs configuration file to **/etc/grafana/grafana.ini**
- Installs systemd service (if systemd is available) name **grafana-server.service**
- The default configuration sets the log file at **/var/log/grafana/grafana.log**
- The default configuration specifies a sqlite3 db at **/var/lib/grafana/grafana.db**
- Installs HTML/JS/CSS and other Grafana files at **/usr/share/grafana**
- Systemd service file & init.d script use env vars on the file on **/etc/default/grafana-server**.

Install Prometheus on Docker & K8s

- Replace [YAML-FILE-PATH] with path of the prometheus.yml file on host.

```
root@ubuntu:~# docker run -d -p 9090:9090 --name prom --volume  
/etc/prometheus.yml:/etc/prometheus/prometheus.yml  
prom/prometheus
```

```
root@ubuntu:~# docker run -d -p 3000:3000 --name
```

- For installing Prometheus/Grafana on Kubernetes needs to use helm:

```
root@ubuntu:~# kubectl create namespace prometheus
```

```
root@ubuntu:~# helm repo add prometheus-community
```

```
https://charts.helm.sh/stable
```

```
root@ubuntu:~# helm repo add prometheus-community https://prometheus-  
community.github.io/helm-charts
```

```
root@ubuntu:~# helm repo update
```

```
root@ubuntu:~# helm install kube-prometheus-stack prometheus-
```

```
community/kube-prometheus-stack --namespace prometheus
```



Install Prometheus Node_Exporter on Linux servers to be monitored



I. Install Prometheus Node Exporter

- We added a system user called **prometheus** whose default group is **prometheus**.

```
root@ubuntu:~# groupadd --system prometheus
root@ubuntu:~# useradd -s /sbin/nologin --system -g prometheus
prometheus
root@ubuntu:~# curl -s
https://api.github.com/repos/prometheus/node_exporter/r
eleases/latest | grep browser_download_url | grep
linux-amd64 | cut -d '"' -f 4 | wget -qi -

root@ubuntu:~# tar -zxvf node_exporter-*linux-
amd64.tar.gz
root@ubuntu:~# cd node_exporter*/
root@ubuntu:~# mv node_exporter /usr/local/bin/
root@ubuntu:~# node_exporter --version
```

II. Configure Node Exporter in systemd

- Collectors are enabled by providing a **--collector.<name>** flag.
- Enabled collectors by default can be disabled by providing a **--no-collector.<name>**

```
root@ubuntu:~# vim /etc/systemd/system/node_exporter.service
```



node_exporter.ser
vice

```
root@ubuntu:~# sudo systemctl enable node_exporter
root@ubuntu:~# sudo systemctl start node_exporter
root@ubuntu:~# ufw allow 9100
```

III. Add Exporter Job to Prometheus

- Last step is to add a job to **Prometheus server** for scraping metrics.
- Edit **/etc/prometheus/prometheus.yml**:

```
root@ubuntu:~# vim /etc/prometheus/prometheus.yml

# Linux Servers
- job_name: apache-linux-server1
  static_configs:
    - targets: ['10.1.10.20:9100']
      labels:
        alias: server1

- job_name: apache-linux-server2
  static_configs:
    - targets: ['10.1.10.21:9100']
      labels:
        alias: server2

root@ubuntu:~# sudo systemctl restart prometheus
```

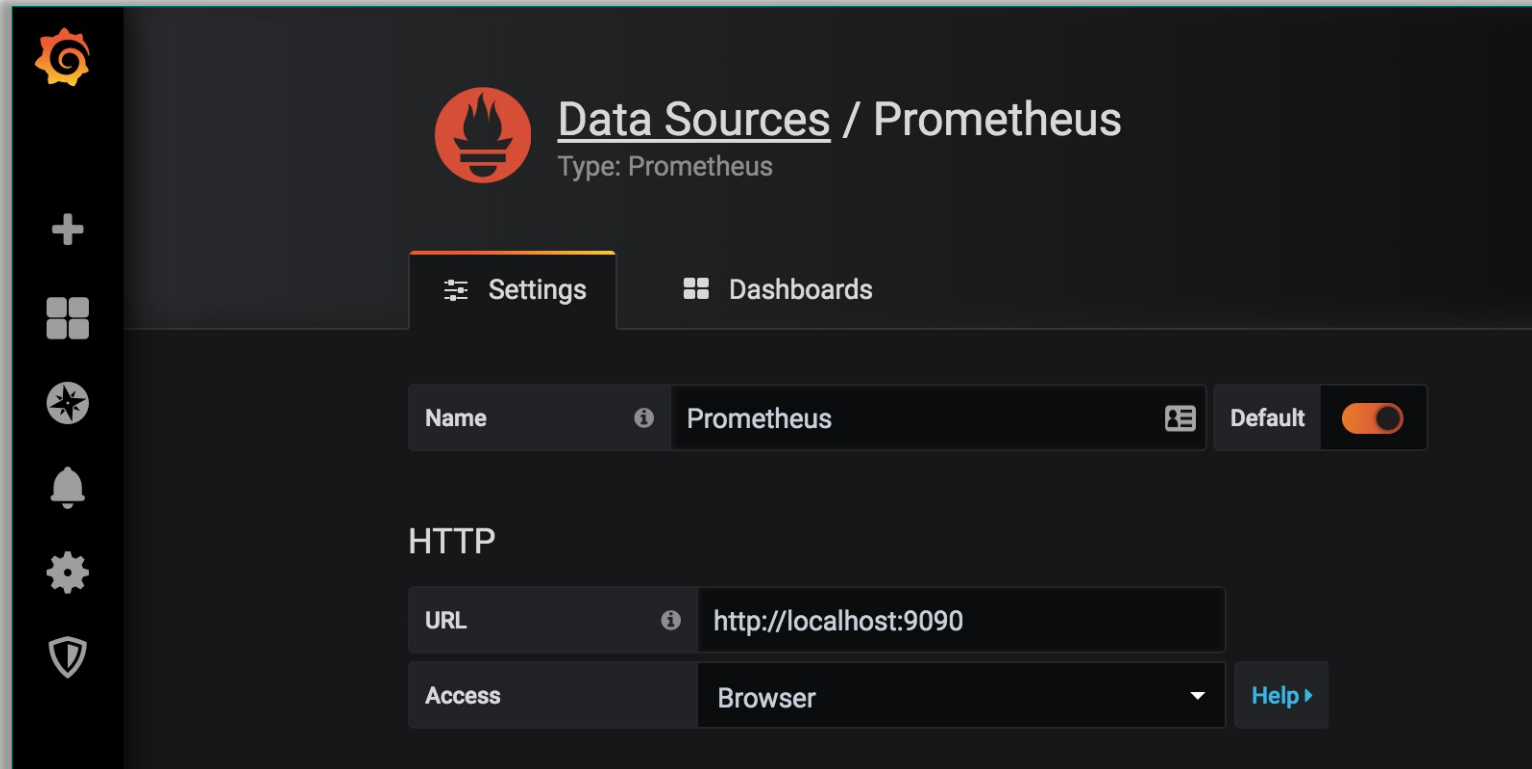
Add Dashboard to Grafana



Creating a Prometheus Data source

- 1) Click on the "**cogwheel**" in the sidebar to open the Configuration menu.
- 2) Click on "**Data Sources**".
- 3) Click on "**Add data source**".
- 4) Select "**Prometheus**" as the type.
- 5) Set the appropriate Prometheus server URL (for example, **http://localhost:9090/**)
- 6) Adjust other data source settings as desired (for example, choosing **right Access** method).
- 7) Click "**Save & Test**" to save the new data source.

Creating a Prometheus Data source



The screenshot shows the Grafana web interface for configuring a Prometheus data source. On the left is a dark sidebar with icons for home, add new, data sources, dashboards, alerts, settings, and help. The main panel has a dark theme. At the top, there's a red Prometheus logo and the title 'Data Sources / Prometheus' with 'Type: Prometheus' below it. Below the title are two tabs: 'Settings' (active) and 'Dashboards'. The 'Settings' tab contains a 'Name' field with the value 'Prometheus' and a 'Default' toggle switch that is turned on. Below this is a section titled 'HTTP' with a 'URL' field containing 'http://localhost:9090' and an 'Access' dropdown menu set to 'Browser'. A 'Help' link is visible next to the 'Access' dropdown.

  **Data Sources / Prometheus**
Type: Prometheus

 Settings  Dashboards

Name  Prometheus  Default ☒

HTTP

URL  http://localhost:9090

Access  Browser [Help >](#)

Creating a Prometheus Graph

❑ Follow the standard way of adding a new Grafana graph. Then:

- 1) Click the **graph** title, then click "**Edit**".
- 2) Under the "**Metrics**" tab, select your Prometheus data source (bottom right).
- 3) Enter any Prometheus expression into the "**Query**" field, while using the "**Metric**" field to lookup metrics via autocompletion.
- 4) To format the legend names of time series, use the "**Legend format**" input. For example, to show only the method and status labels of a returned query result, separated by a dash, you could use the legend format string `{{method}} - {{status}}`.
- 5) Tune other graph settings until you have a working graph.

Importing Pre-Built Dashboards from Grafana.com

- ❑ **Grafana.com** maintains [a collection of shared dashboards](#) which can be downloaded and used with standalone instances of Grafana. Use the **Grafana.com "Filter"** option to browse dashboards for the "Prometheus" data source only.
- ❑ You must currently manually edit the downloaded **JSON** files and correct the datasource: **entries to reflect the Grafana data source name which you chose for your Prometheus server**. Use the "Dashboards" → "Home" → "Import" option to import the edited dashboard file into your Grafana install.

Add Dashboard to Grafana

- With Prometheus data source added to Grafana, Import Apache Grafana Dashboard by navigating to **Dashboard > Import**. Use **159** for Grafana Dashboard ID.

❑ Node environment monitoring:

- o Recommended: <https://grafana.com/grafana/dashboards/159>
- o <https://grafana.com/dashboards/1860>
- o <https://grafana.com/dashboards/3662>
- o <https://grafana.com/dashboards/8919>
- o <https://github.com/arashforoughi/grafana-dashboards>

Add Dashboard to Grafana

❑ Docker and System Monitoring:

- o Recommended: <https://grafana.com/grafana/dashboards/14282>
- o <https://grafana.com/grafana/dashboards/179>
- o <https://grafana.com/grafana/dashboards/11600>
- o <https://grafana.com/grafana/dashboards/193>

❑ Kubernetes Cluster Monitoring:

- o Recommended: <https://grafana.com/grafana/dashboards/315>
- o <https://grafana.com/grafana/dashboards/6336>
- o <https://grafana.com/grafana/dashboards/395>
- o <https://grafana.com/grafana/dashboards/7249>



Container Monitoring with Prometheus & cAdvisor



Container Monitoring with cAdvisor

- **cAdvisor** is a daemon that runs into a container that exports statistics regarding containers resource usage from the host machine.
- It analyzes even container processes within. It has made by **Google** and it's Docker native.
- We can use it as **stand-alone** or **export** its statistics to Prometheus.
- **cAdvisor** can be run on two way:
 1. Docker
 2. Docker-Compose (optional)

```
- job_name: 'cadvisor'  
  static_configs:  
    - targets:  
      ['cadvisor:8080']  
      labels:  
        alias: 'cadvisor'
```



cAdvisor

Running cAdvisor in Docker

- Start cAdvisor image as a container like below:

```
root@ubuntu:~# docker run --volume /:/rootfs:ro --volume  
/var/run:/var/run:ro --volume /sys:/sys:ro --volume  
/var/lib/docker/:/var/lib/docker:ro --volume /dev/disk/:/dev/disk:ro  
--publish 8080:8080 -dit --name cadvisor gcr.io/cadvisor/cadvisor
```

<https://www.metricfire.com/blog/monitoring-docker-containers-with-cadvisor/>

```
version: '3.4'
services:
  prometheus:
    image: 'prom/prometheus:latest'
    container_name: prometheus
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    commands:
      - '--config.file=/etc/prometheus/prometheus.yml'
    ports:
      - '9090:9090'

  cadvisor:
    image: 'google/cadvisor:latest'
    container_name: cadvisor
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker:/var/lib/docker:ro
      - /dev/disk:/dev/disk:ro
    ports:
      - '8080:8080'
```

Running cAdvisor with Docker Compose



Kubernetes Monitoring with Prometheus



What is Kube State Metrics?

- **Kube State metrics** is a service that talks to the **Kubernetes API server** to get all details about all **API objects** like deployments, pods, daemonsets, Statefulsets, etc.
- It produces **metrics** in Prometheus format with the stability as the Kubernetes API.
- **Prometheus** can scrape metrics exposed by Kube state metrics.
- Some of important metrics you can get from Kube state metrics:
 - Node status, node capacity (CPU and memory)
 - Replica-set compliance (desired/available/unavailable/updated status of replicas per deployment)
 - Pod status (waiting, running, ready, etc)
 - PV, PVC metrics
 - Resource requests and limits.
 - Job & Cronjob metrics

Kube State Metrics Setup

- **Kube state metrics** is available as a **public docker image**. You will have to deploy following Kubernetes objects for Kube state metrics to work:
 - 1) A **Service Account**
 - 2) **Cluster Role** – For kube state metrics to access all the Kubernetes API objects.
 - 3) **Cluster Role Binding** – Binds the service account with the cluster role.
 - 4) Kube State Metrics **Deployment**
 - 5) **Service** – To expose the metrics
- All the above Kube state metrics objects will be deployed in the **kube-system namespace**

Kube State Metrics Setup

- All the deployment objects are available in Github:

```
root@ubuntu:~# git clone https://github.com/devopscube/kube-state-metrics-configs.git
```

```
root@ubuntu:~# kubectl apply -f kube-state-metrics-configs/
```

```
root@ubuntu:~# kubectl get deployments kube-state-metrics -n kube-system
```

```
root@ubuntu:~# vim /etc/prometheus/prometheus.yml
```

```
- job_name: 'kube-state-metrics'
```

```
  static_configs:
```

```
    - targets: [192.168.44.25:8080']
```

Thank you.

